

Python Project Setup with uv – Step-by-Step Guide for Windows & macOS

Install uv and Python 3.12, create a project and a virtual environment, install the libraries from requirements.txt with uv, then run, stop and manage dashboards.

Your computer	Go to
Windows 10 / 11	Part A (commands typed in PowerShell)
macOS (Intel or Apple Silicon)	Part B (commands typed in Terminal)
Change libraries, renew requirements.txt	Steps 12–14: A5 (Windows) · B5 (macOS)
Dashboard code to paste	Appendix 1 and Appendix 2

Contents

Part A. Windows	3
A1. Install uv (one time only).....	3
A2. Install Python 3.12 (one time only)	3
A3. Create the project (one time only).....	3
A4. Create the virtual environment (one time only)	4
A5. Install the libraries with requirements.txt	4
A6. Create and run a dashboard.....	5
A7. Every time you want to open the dashboard	6
A8. Stop	6
A9. Run a different dashboard in the same project	6
Part B. macOS	7
B1. Install uv (one time only).....	7
B2. Install Python 3.12 (one time only)	7
B3. Create the project (one time only).....	7
B4. Create the virtual environment (one time only)	8
B5. Install the libraries with requirements.txt	8
B6. Create and run a dashboard.....	9
B7. Every time you want to open the dashboard.....	10
B8. Stop	10
B9. Run a different dashboard in the same project	10
Quick reference	11
Open the dashboard: 3 commands	12
pip version vs uv version	12
Appendix 1. expenses.py	13
Appendix 2. sales.py	16

Part A. Windows

A1. Install uv (one time only)

Step 1. Open PowerShell

1. Press the **Windows** key, type **powershell**, press **Enter**.

Step 2. Install uv

```
powershell -ExecutionPolicy Bypass -c "irm https://astral.sh/uv/install.ps1 | iex"
```

This is the official uv installer. It does not need Python or **pip**.

Step 3. Restart PowerShell

1. Close the PowerShell window.
2. Press the **Windows** key, type **powershell**, press **Enter**.

Step 4. Check uv

```
uv --version
```

YOU SHOULD SEE

```
uv 0.x.x
```

A2. Install Python 3.12 (one time only)

Step 5. Install Python with uv

```
uv python install 3.12
```

YOU SHOULD SEE

```
Installed Python 3.12.x in ...
```

uv downloads Python for you. You do not need the installer from python.org.

Step 6. Check Python

```
uv python list --only-installed
```

YOU SHOULD SEE

```
cpython-3.12.x-windows-x86_64-none C:\Users\<your-name>\AppData\Roaming\uv\python\...
```

A3. Create the project (one time only)

Step 7. Create the project folder

```
mkdir $HOME\Downloads\code-practice  
cd $HOME\Downloads\code-practice
```

If **mkdir** shows an error because the folder already exists, just continue.

Step 8. Create the uv project

```
uv init --bare --python 3.12
```

YOU SHOULD SEE

```
Initialized project `code-practice`
```

This creates `pyproject.toml`, the file where uv writes down which packages the project uses.

A4. Create the virtual environment (one time only)

Step 9. Create the virtual environment

```
uv venv --python 3.12
```

YOU SHOULD SEE

```
Using CPython 3.12.x
Creating virtual environment at: .venv
Activate with: .venv\Scripts\activate
```

Step 10. Activate the virtual environment

```
.\.venv\Scripts\Activate.ps1
```

YOU SHOULD SEE

```
(code-practice) PS C:\Users\<your-name>\Downloads\code-practice>
```

Step 11. Check the Python version inside the environment

```
python --version
```

YOU SHOULD SEE

```
Python 3.12.x
```

A5. Install the libraries with requirements.txt

Step 12. Making requirements.txt and share

```
notepad requirements.txt
```

1. When Notepad asks to create a new file, click **Yes**.
2. Type the libraries below, one per line:

```
numpy
pandas
matplotlib
seaborn
streamlit
yfinance
```

3. Save with **Ctrl + S**.
4. Close Notepad.

Install everything lightning-fast using:

```
uv pip install -r requirements.txt
```

(Or standard pip: `pip install -r requirements.txt`)

Standard pip only works in an environment made with `py -3.12 -m venv venv`. An environment made with `uv venv` (Step 9) has no pip inside, so use the `uv pip` command above.

Step 13. Adding New Libraries with `uv`

If you are working inside a project environment managed by `uv`:

Option A: Project / Virtual Environment Mode

If you are managing your project using `uv` (with a `pyproject.toml` file):

```
# Add a single library
uv add plotly

# Add multiple libraries
uv add scikit-learn scipy
```

This automatically installs the package and updates your dependency configuration.

Lines that start with `#` are comments. Type only the lines that start with `uv`.

Option B: Standard Virtual Environment (`pip`-style)

If you simply created a standard virtual environment and want to quickly install packages:

```
uv pip install plotly scikit-learn
```

Step 14. Renewing `requirements.txt` with `uv` (later)

If you are currently inside an active virtual environment and want to list installed packages:

```
uv pip freeze > requirements.txt
```

A6. Create and run a dashboard

Step 15. Open a new file in Notepad

```
notepad expenses.py
```

When Notepad asks to create a new file, click **Yes**.

Step 16. Paste the code and save

1. Copy all of the code in **Appendix 1**.
2. Paste it into Notepad with **Ctrl + V**.
3. Save with **Ctrl + S**.
4. Close Notepad.

Step 17. Check the files exist

```
ls
```

YOU SHOULD SEE

```
.venv
expenses.py
pyproject.toml
requirements.txt
```

`uv.lock` is also listed if you used `uv add` in Step 13.

Step 18. Run the dashboard

```
streamlit run expenses.py
```

1. If it asks for **Email**, leave it blank and press **Enter**.

2. Your browser opens **http://localhost:8501**.
3. If the browser does not open, copy **http://localhost:8501** into the browser.

A7. Every time you want to open the dashboard

1. Press the **Windows** key, type **powershell**, press **Enter**.
2. Run these three commands:

```
cd $HOME\Downloads\code-practice
.\.venv\Scripts\Activate.ps1
streamlit run expenses.py
```

A8. Stop

To do this	Do this
Stop the dashboard	Click the PowerShell window running it, press Ctrl + C .
Close the virtual environment	Type deactivate .
Stop everything	Close the PowerShell window.

NOTE · Closing the browser tab does not stop the dashboard

Use **Ctrl + C** in PowerShell.

A9. Run a different dashboard in the same project

1. Stop the running dashboard with **Ctrl + C**.
2. Create the new file, for example **notepad sales.py**, paste the code from **Appendix 2**, save.
3. Run it:

```
streamlit run sales.py
```

No need to activate again while **(code-practice)** is still showing. No need to install anything else: the libraries from **requirements.txt** are already installed.

Part B. macOS

B1. Install uv (one time only)

Step 1. Open Terminal

1. Press **Cmd + Space**, type **terminal**, press **Enter**.
2. If the line starts with **(base)**, run **conda deactivate**.

Step 2. Install uv

```
curl -LsSf https://astral.sh/uv/install.sh | sh
```

This is the official uv installer. It does not need Python or **pip**.

Step 3. Restart Terminal

1. Press **Cmd + Q** to quit Terminal.
2. Press **Cmd + Space**, type **terminal**, press **Enter**.

Step 4. Check uv

```
uv --version
```

YOU SHOULD SEE

```
uv 0.x.x
```

CHECK · zsh: command not found: uv

Quit Terminal, open it again and repeat Step 4. If it still fails, repeat Step 2.

B2. Install Python 3.12 (one time only)

Step 5. Install Python with uv

```
uv python install 3.12
```

YOU SHOULD SEE

```
Installed Python 3.12.x in ...
```

uv downloads Python for you. You do not need the installer from python.org, and there is no Install Certificates step.

Step 6. Check Python

```
uv python list --only-installed
```

YOU SHOULD SEE

```
cpython-3.12.x-macos-... /Users/<your-name>/.local/share/uv/python/...
```

B3. Create the project (one time only)

Step 7. Create the project folder

```
mkdir -p ~/Downloads/code-practice
cd ~/Downloads/code-practice
```

Step 8. Create the uv project

```
uv init --bare --python 3.12
```

YOU SHOULD SEE

```
Initialized project `code-practice`
```

This creates `pyproject.toml`, the file where uv writes down which packages the project uses.

B4. Create the virtual environment (one time only)

Step 9. Create the virtual environment

```
uv venv --python 3.12
```

YOU SHOULD SEE

```
Using CPython 3.12.x
Creating virtual environment at: .venv
Activate with: source .venv/bin/activate
```

Step 10. Activate the virtual environment

```
source .venv/bin/activate
```

YOU SHOULD SEE

```
(code-practice) your-name@MacBook code-practice %
```

Step 11. Check the Python version inside the environment

```
python --version
```

YOU SHOULD SEE

```
Python 3.12.x
```

B5. Install the libraries with requirements.txt

Step 12. Making requirements.txt and share

```
nano requirements.txt
```

1. Type the libraries below, one per line:

```
numpy
pandas
matplotlib
seaborn
streamlit
yfinance
```

2. Save: press **Control + O**, then **Enter**.

3. Exit: press **Control + X**.

Install everything lightning-fast using:

```
uv pip install -r requirements.txt
```

(Or standard pip: `pip install -r requirements.txt`)

Standard pip only works in an environment made with `python3 -m venv venv`. An environment made with `uv venv` (Step 9) has no pip inside, so use the `uv pip` command above.

Step 13. Adding New Libraries with `uv`

If you are working inside a project environment managed by `uv`:

Option A: Project / Virtual Environment Mode

If you are managing your project using `uv` (with a `pyproject.toml` file):

```
# Add a single library
uv add plotly

# Add multiple libraries
uv add scikit-learn scipy
```

This automatically installs the package and updates your dependency configuration.

Lines that start with `#` are comments. Type only the lines that start with `uv`. In Terminal, typing a `#` line gives an error.

Option B: Standard Virtual Environment (`pip`-style)

If you simply created a standard virtual environment and want to quickly install packages:

```
uv pip install plotly scikit-learn
```

Step 14. Renewing `requirements.txt` with `uv` (later)

If you are currently inside an active virtual environment and want to list installed packages:

```
uv pip freeze > requirements.txt
```

B6. Create and run a dashboard

Step 15. Open a new file in nano

```
nano expenses.py
```

Step 16. Paste the code and save

1. Copy all of the code in **Appendix 1**.
2. Paste it into nano with **Cmd + V**.
3. Save: press **Control + O**, then **Enter**.
4. Exit: press **Control + X**.

Step 17. Check the files exist

```
ls
```

YOU SHOULD SEE

```
expenses.py  pyproject.toml  requirements.txt
```

`.venv` is hidden on macOS, so `ls` does not show it. That is normal.

`uv.lock` is also listed if you used `uv add` in Step 13.

Step 18. Run the dashboard

```
streamlit run expenses.py
```

1. If it asks for **Email**, leave it blank and press **Enter**.
2. Your browser opens **http://localhost:8501**.
3. If the browser does not open, copy **http://localhost:8501** into the browser.

B7. Every time you want to open the dashboard

1. Press **Cmd + Space**, type **terminal**, press **Enter**.
2. If the line starts with **(base)**, run **conda deactivate**.
3. Run these three commands:

```
cd ~/Downloads/code-practice
source .venv/bin/activate
streamlit run expenses.py
```

B8. Stop

To do this	Do this
Stop the dashboard	Click the Terminal window running it, press Control + C .
Close the virtual environment	Type deactivate .
Stop everything	Press Cmd + Q to quit Terminal. If asked, click Terminate .

NOTE · Control + C stops the dashboard

Cmd + C only copies text.

B9. Run a different dashboard in the same project

1. Stop the running dashboard with **Control + C**.
2. Create the new file, for example **nano sales.py**, paste the code from **Appendix 2**, save, exit.
3. Run it:

```
streamlit run sales.py
```

No need to activate again while **(code-practice)** is still showing. No need to install anything else: the libraries from **requirements.txt** are already installed.

Quick reference

Task	Windows (PowerShell)	macOS (Terminal)
Install uv (once)	<code>powershell -ExecutionPolicy ByPass -c "irm https://astral.sh/uv/install.ps1 iex"</code>	<code>curl -LsSf https://astral.sh/uv/install.sh sh</code>
Install Python (once)	<code>uv python install 3.12</code>	<code>uv python install 3.12</code>
Go to project	<code>cd \$HOME\Downloads\code-practice</code>	<code>cd ~/Downloads/code-practice</code>
Create project	<code>uv init --bare --python 3.12</code>	<code>uv init --bare --python 3.12</code>
Create environment	<code>uv venv --python 3.12</code>	<code>uv venv --python 3.12</code>
Activate	<code>.\.venv\Scripts\Activate.ps1</code>	<code>source .venv/bin/activate</code>
Allow activation (once, only if blocked)	<code>Set-ExecutionPolicy - ExecutionPolicy RemoteSigned - Scope CurrentUser</code>	Not needed
Check Python	<code>python --version</code>	<code>python --version</code>
Create requirements file	<code>notepad requirements.txt</code>	<code>nano requirements.txt</code>
Install libraries	<code>uv pip install -r requirements.txt</code>	<code>uv pip install -r requirements.txt</code>
Add a library	<code>uv add plotly</code>	<code>uv add plotly</code>
Add several libraries	<code>uv add scikit-learn scipy</code>	<code>uv add scikit-learn scipy</code>
Install, pip-style	<code>uv pip install plotly scikit- learn</code>	<code>uv pip install plotly scikit- learn</code>
Renew requirements.txt	<code>uv pip freeze > requirements.txt</code>	<code>uv pip freeze > requirements.txt</code>
Create / edit file	<code>notepad expenses.py</code>	<code>nano expenses.py</code>
List files	<code>ls</code>	<code>ls</code>
Run dashboard	<code>streamlit run expenses.py</code>	<code>streamlit run expenses.py</code>
Run without activating	<code>uv run streamlit run expenses.py</code>	<code>uv run streamlit run expenses.py</code>
Stop dashboard	Ctrl + C	Control + C
Deactivate	<code>deactivate</code>	<code>deactivate</code>
Delete environment	<code>Remove-Item -Recurse -Force .venv</code>	<code>rm -rf .venv</code>
Rebuild environment (after deleting)	<code>uv venv --python 3.12 then uv pip install -r requirements.txt</code>	<code>uv venv --python 3.12 then uv pip install -r requirements.txt</code>

Open the dashboard: 3 commands

Windows

```
cd $HOME\Downloads\code-practice
.\.venv\Scripts\Activate.ps1
streamlit run expenses.py
```

Stop: Ctrl + C

macOS

```
cd ~/Downloads/code-practice
source .venv/bin/activate
streamlit run expenses.py
```

Stop: Control + C

pip version vs uv version

Task	pip version	uv version
Install Python	Installer from python.org	<code>uv python install 3.12</code>
Create environment	<code>py -3.12 -m venv venv</code>	<code>uv init --bare --python 3.12</code> then <code>uv venv --python 3.12</code>
Environment folder	<code>venv</code>	<code>.venv</code>
Prompt after activate	<code>(venv)</code>	<code>(code-practice)</code>
Install libraries	<code>pip install -r requirements.txt</code>	<code>uv pip install -r requirements.txt</code>
Add a library	<code>pip install plotly</code>	<code>uv add plotly</code>
Renew requirements.txt	<code>pip freeze > requirements.txt</code>	<code>uv pip freeze > requirements.txt</code>
Run dashboard	<code>streamlit run expenses.py</code>	<code>streamlit run expenses.py</code> (same)

Appendix 1. expenses.py

Copy everything from `import streamlit as st` to the last line. Keep the spaces at the start of each line.

```
import streamlit as st
import calendar
import csv
import io
import random
from datetime import date

st.set_page_config(page_title="Expense Tracker", layout="wide")

CATEGORIES = ["Rent", "Food", "Transport", "Shopping", "Entertainment", "Bills",
              "Health"]

# ----- Sample data: 6 months of personal expenses -----
@st.cache_data
def load_data():
    rnd = random.Random(7)
    # category: (min times per month, max times, min amount, max amount)
    plan = {
        "Food": (25, 35, 40_000, 250_000),
        "Transport": (15, 25, 20_000, 150_000),
        "Shopping": (3, 6, 150_000, 1_500_000),
        "Entertainment": (2, 5, 100_000, 600_000),
        "Bills": (2, 3, 300_000, 900_000),
        "Health": (0, 2, 100_000, 800_000),
    }
    methods = ["Cash", "Card", "E-wallet"]
    rows = []
    for month in range(3, 9):
        # March to August 2026
        last_day = calendar.monthrange(2026, month)[1]
        rows.append({"date": date(2026, month, 1), "category": "Rent",
                    "amount": 6_000_000, "method": "Bank transfer"})
        for category, (lo, hi, a_min, a_max) in plan.items():
            for _ in range(rnd.randint(lo, hi)):
                rows.append({
                    "date": date(2026, month, rnd.randint(1, last_day)),
                    "category": category,
                    "amount": int(rnd.uniform(a_min, a_max) // 1000 * 1000),
                    "method": rnd.choice(methods),
                })
    rows.sort(key=lambda r: r["date"])
    return rows

def vnd(x):
    return f"{x:,.0f} VND"

def month_key(row):
    return row["date"].strftime("%Y-%m")

def total_of(rows):
    return sum(r["amount"] for r in rows)
```

```

def sum_by(rows, field):
    totals = {}
    for r in rows:
        totals[r[field]] = totals.get(r[field], 0) + r["amount"]
    return dict(sorted(totals.items(), key=lambda kv: kv[1], reverse=True))

data = load_data()
all_months = sorted({month_key(r) for r in data})

# ----- Sidebar -----
with st.sidebar:
    st.header("Settings")
    month = st.selectbox("Month", all_months, index=len(all_months) - 1)
    budget = st.number_input("Monthly budget (VND)", min_value=1_000_000,
                             max_value=50_000_000, value=20_000_000, step=500_000)
    method = st.radio("Payment method", ["All", "Cash", "Card", "E-wallet", "Bank
transfer"])
    st.caption("Sample data, not real expenses.")

chosen = data if method == "All" else [r for r in data if r["method"] == method]
this_month = [r for r in chosen if month_key(r) == month]

st.title("Personal Expense Tracker")
st.caption(f"Month: {month} · Payment method: {method}")

if not this_month:
    st.info("No expenses match this month and payment method.")
    st.stop()

# ----- Key numbers -----
total = total_of(this_month)
by_category = sum_by(this_month, "category")

c1, c2, c3, c4 = st.columns(4)
c1.metric("Total spent", vnd(total))
c2.metric("Budget left", vnd(budget - total))
c3.metric("Transactions", f"{len(this_month)}")
c4.metric("Biggest category", next(iter(by_category)))

# ----- Budget bar -----
used = total / budget
st.progress(min(used, 1.0), text=f"{used:.0%} of budget used")
if used > 1:
    st.error(f"Over budget by {vnd(total - budget)}")
elif used > 0.8:
    st.warning("More than 80% of the budget is used.")
else:
    st.success("Spending is within budget.")

st.divider()

# ----- Charts -----
left, right = st.columns(2)

with left:
    st.subheader("Spending by category")
    st.bar_chart({"Category": list(by_category), "Amount": list(by_category.values())},
                  x="Category", y="Amount", horizontal=True, height=300)

with right:

```

```

st.subheader("Running total vs budget")
year, month_number = int(month[:4]), int(month[5:])
last_day = calendar.monthrange(year, month_number)[1]
per_day = {}
for r in this_month:
    per_day[r["date"].day] = per_day.get(r["date"].day, 0) + r["amount"]
running, spent = [], 0
for day in range(1, last_day + 1):
    spent += per_day.get(day, 0)
    running.append(spent)
st.line_chart(
    {"Day": [date(year, month_number, d) for d in range(1, last_day + 1)],
     "Spent": running,
     "Budget": [budget] * last_day},
    x="Day", y=["Spent", "Budget"], height=300,
)

st.subheader("Last 6 months by category")
monthly = {"Month": all_months}
for category in CATEGORIES:
    monthly[category] = [
        total_of([r for r in chosen if month_key(r) == m and r["category"] == category])
        for m in all_months
    ]
st.bar_chart(monthly, x="Month", y=CATEGORIES, height=320)

# ----- What-if saving -----
st.subheader("What if I spend less?")
cut = st.slider("Cut Food, Shopping and Entertainment by (%)", 0, 50, 20, step=5)
flexible = total_of([r for r in this_month
                     if r["category"] in ("Food", "Shopping", "Entertainment")])
saving = flexible * cut / 100
st.write(f"You would save about **{vnd(saving)}** this month, "
        f"or **{vnd(saving * 12)}** in a year.")

# ----- Details -----
columns = {
    "date": st.column_config.DateColumn("Date", format="DD MMM"),
    "category": "Category",
    "amount": st.column_config.NumberColumn("Amount (VND)", format="%d"),
    "method": "Method",
}

with st.expander("Top 10 biggest expenses"):
    biggest = sorted(this_month, key=lambda r: r["amount"], reverse=True)[:10]
    st.dataframe(biggest, hide_index=True, column_config=columns)

with st.expander("All transactions this month"):
    st.dataframe(this_month, hide_index=True, height=350, column_config=columns)

    buffer = io.StringIO()
    writer = csv.DictWriter(buffer, fieldnames=["date", "category", "amount", "method"])
    writer.writeheader()
    writer.writerows(this_month)
    st.download_button("Download CSV", buffer.getvalue().encode("utf-8-sig"),
                       file_name="expenses.csv", mime="text/csv")

```

Appendix 2. sales.py

Copy everything from `import streamlit as st` to the last line. Keep the spaces at the start of each line.

```
import streamlit as st
import csv
import io
import math
import random
from datetime import date, timedelta

st.set_page_config(page_title="Online Store Sales", layout="wide")

CHANNELS = ["Website", "Shopee", "Lazada", "TikTok Shop", "Facebook"]
PRODUCTS = {
    "T-shirt": 180_000,
    "Hoodie": 450_000,
    "Sneakers": 890_000,
    "Backpack": 520_000,
    "Cap": 150_000,
    "Socks": 60_000,
}

# ----- Sample data: 6 months of online orders -----
@st.cache_data
def load_data():
    rnd = random.Random(11)
    first, last = date(2026, 3, 1), date(2026, 8, 31)
    span = (last - first).days + 1

    # How popular each channel is, and how fast it is growing
    weight = {"Website": 1.0, "Shopee": 1.7, "Lazada": 0.9,
              "TikTok Shop": 1.4, "Facebook": 0.6}
    growth = {"Website": 1.10, "Shopee": 1.35, "Lazada": 0.95,
              "TikTok Shop": 1.90, "Facebook": 1.05}

    rows = []
    for i in range(span):
        day = first + timedelta(days=i)
        season = 1.0 + 0.25 * math.sin(i / 30)          # slow ups and downs
        weekend = 1.30 if day.weekday() >= 5 else 1.0
        for channel in CHANNELS:
            ramp = 1 + (growth[channel] - 1) * (i / span)
            mean = 8 * weight[channel] * ramp * season * weekend
            n_orders = max(0, round(rnd.gauss(mean, mean ** 0.5)))
            for _ in range(n_orders):
                product = rnd.choice(list(PRODUCTS))
                units = rnd.randint(1, 3)
                rows.append({
                    "order_id": f"ORD-{10_000 + len(rows)}",
                    "date": day,
                    "channel": channel,
                    "product": product,
                    "units": units,
                    "revenue": units * PRODUCTS[product],
                })
    return rows
```

```

def vnd(x):
    return f"{x / 1_000_000:,.1f}M VND"

def total_of(rows, field="revenue"):
    return sum(r[field] for r in rows)

def sum_by(rows, field, value="revenue"):
    totals = {}
    for r in rows:
        totals[r[field]] = totals.get(r[field], 0) + r[value]
    return dict(sorted(totals.items(), key=lambda kv: kv[1], reverse=True))

data = load_data()
first_day = data[0]["date"]
last_day = data[-1]["date"]

# ----- Sidebar -----
with st.sidebar:
    st.header("Filters")
    picked = st.date_input("Date range", value=(first_day, last_day),
                           min_value=first_day, max_value=last_day)
    channels = st.multiselect("Sales channel", CHANNELS, default=CHANNELS)
    compare = st.toggle("Compare with previous period", value=True)
    st.divider()
    st.caption("Sample data, not real sales.")

# date_input gives one date while you are still picking, two when you are done
if isinstance(picked, (list, tuple)):
    start, end = picked[0], picked[-1]
else:
    start = end = picked

view = [r for r in data if start <= r["date"] <= end and r["channel"] in channels]

st.title("Online Store Sales")
st.caption(f"{start:%d %b %Y} - {end:%d %b %Y} · {len(channels)} of {len(CHANNELS)} channels")

if not view:
    st.info("No orders match these filters. Pick at least one channel.")
    st.stop()

# ----- Key numbers -----
n_days = (end - start).days + 1
prev_start = start - timedelta(days=n_days)
prev_end = start - timedelta(days=1)
before = [r for r in data if prev_start <= r["date"] <= prev_end and r["channel"] in channels]

def delta(now, then):
    # No arrow when the toggle is off, or when there is nothing to compare with
    if not compare or then == 0:
        return None
    return f"{{(now - then) / then * 100:+.1f}}%"

revenue_now = total_of(view)
units_now = total_of(view, "units")

```

```

c1, c2, c3, c4 = st.columns(4)
c1.metric("Revenue", vnd(revenue_now), delta(revenue_now, total_of(before)))
c2.metric("Orders", f"{len(view):,}", delta(len(view), len(before)))
c3.metric("Units sold", f"{units_now:,.0f}", delta(units_now, total_of(before, "units")))
c4.metric("Average order value", f"{revenue_now / len(view):,.0f} VND")

if compare:
    st.caption("Arrows compare with the period of the same length just before this
one.")

st.divider()

# ----- Tabs -----
tab1, tab2, tab3 = st.tabs(["Trend", "Channels and products", "Orders"])

days = [start + timedelta(days=i) for i in range(n_days)]

with tab1:
    st.subheader("Revenue per day")
    per_day = sum_by(view, "date")
    st.area_chart({"Day": days, "Revenue": [per_day.get(d, 0) for d in days]},
                  x="Day", y="Revenue", height=320)

    st.subheader("Revenue per day, split by channel")
    split = {"Day": days}
    for channel in channels:
        totals = sum_by([r for r in view if r["channel"] == channel], "date")
        split[channel] = [totals.get(d, 0) for d in days]
    st.line_chart(split, x="Day", y=channels, height=320)

with tab2:
    left, right = st.columns(2)

    with left:
        st.subheader("Revenue by channel")
        by_channel = sum_by(view, "channel")
        st.bar_chart({"Channel": list(by_channel), "Revenue":
list(by_channel.values())},
                    x="Channel", y="Revenue", horizontal=True, height=300)

    with right:
        st.subheader("Revenue by product")
        by_product = sum_by(view, "product")
        st.bar_chart({"Product": list(by_product), "Revenue":
list(by_product.values())},
                    x="Product", y="Revenue", horizontal=True, height=300)

    st.subheader("Product table")
    # One small line chart inside each row: weekly revenue for that product
    weeks = sorted({r["date"] - timedelta(days=r["date"].weekday()) for r in view})
    table = []
    for product, revenue in by_product.items():
        lines = [r for r in view if r["product"] == product]
        weekly = sum_by(lines, "date")
        trend = []
        for week_start in weeks:
            week_days = [week_start + timedelta(days=i) for i in range(7)]
            trend.append(sum(weekly.get(d, 0) for d in week_days))
        table.append({
            "product": product,
            "units": total_of(lines, "units"),
            "revenue": revenue,

```

```

        "orders": len(lines),
        "share": revenue / total_of(view) * 100,
        "trend": trend,
    })

st.dataframe(
    table,
    hide_index=True,
    column_config={
        "product": "Product",
        "units": st.column_config.NumberColumn("Units", format="%d"),
        "revenue": st.column_config.NumberColumn("Revenue (VND)", format="%d"),
        "orders": st.column_config.NumberColumn("Orders", format="%d"),
        "share": st.column_config.ProgressColumn(
            "Share of revenue", min_value=0,
            max_value=max(row["share"] for row in table), format="%.1f%"),
        "trend": st.column_config.LineChartColumn("Weekly trend", y_min=0),
    },
)

with tab3:
    st.subheader("Order lines")
    st.write(f"Showing {len(view):,} rows.")

    newest_first = sorted(view, key=lambda r: r["date"], reverse=True)
    st.dataframe(
        newest_first,
        hide_index=True,
        height=400,
        column_config={
            "order_id": "Order",
            "date": st.column_config.DateColumn("Date", format="DD MMM YYYY"),
            "channel": "Channel",
            "product": "Product",
            "units": st.column_config.NumberColumn("Units"),
            "revenue": st.column_config.NumberColumn("Revenue (VND)", format="%d"),
        },
    )

    buffer = io.StringIO()
    writer = csv.DictWriter(buffer, fieldnames=list(view[0]))
    writer.writeheader()
    writer.writerows(newest_first)
    st.download_button("Download CSV", buffer.getvalue().encode("utf-8-sig"),
                        file_name="online_store_sales.csv", mime="text/csv")

```